TATA CONSULTANCY SERVICES
Experience certainty.

Kubernetes Walkthrough
Part I.

Ratheesh G Variar

ratheesh.variar@tcs.com

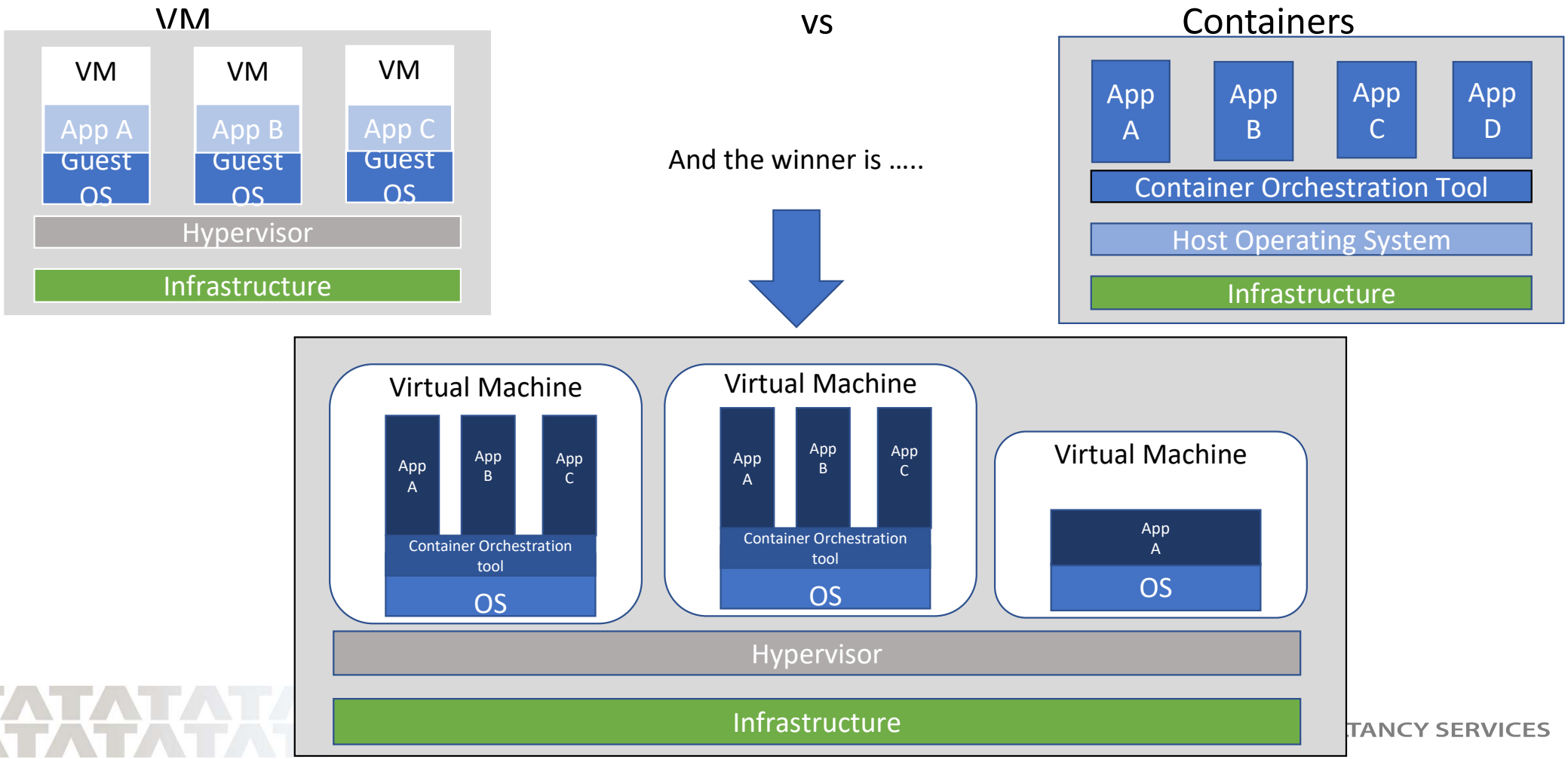
Agenda

- Containerization.
- Need for Container Orchestration Tool.
- Kubernetes(K8) Architecture.
- Pods in K8.
- Volumes in K8.

Containerization

- A method of virtualization where application and its dependencies packed into a single isolated package.
- Application runs quickly, reliably and can be ported easily from one computing environment to another.
- Benefits include :-
  - > Suited for agile environment
  - > Facilitates approaches such microservices , CI/CD
  - > Eliminates platform inconsistencies – unpredictability.

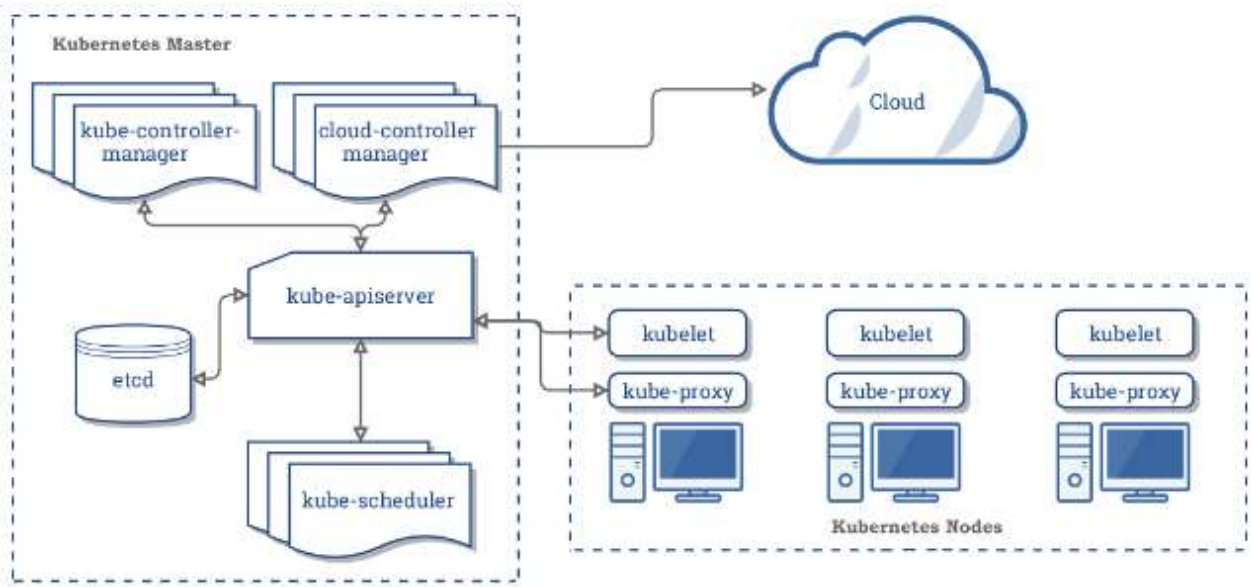
VM and Containers



Need for Container Orchestration Tool

- Various system available to create containers, Docker undoubtedly most efficient and widely used.
- Container Orchestration Tool :-
  - More the number of containers in application greater more complex it becomes to handle them.
  - Deploying connecting containers across multiple host
  - Scaling, deploying without down time.
  - Options available Kubernetes, Docker Swarm

K8 Architecture



Master Explained

- All components of Master are together called as control plane.
- **Kube-apiserver :-**
  The Kubernetes API server validates and configures data for the api objects.
- **etcd :-**
  Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data.
- **kube-scheduler :-**
  Watches newly created pods that have no node assigned and selects a node for them to run on.
- **Kube-controller manager:-**
  Component on the master that runs controllers such as node , endpoint, endpoint, service account – token controller.
- **cloud-controller manager :-**
  Kubernetes v1.6 introduced a new binary called cloud-controller-manager. cloud-controller -manager is a daemon that embeds cloud-specific control loops. These cloud-specific control loops were originally in the kube-controller-manager.

Node Explained

- Node components run on every node, maintaining running pods and providing the Kubernetes runtime environment.
- **Kubelet :-**

  The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. The kubelet doesn't manage containers which were not created by Kubernetes.
- **Kube-proxy**

  kube-proxy is a network proxy that runs on each node in the cluster, implementing part of the Kubernetes Service concept. it is used to connect the application to the external world/environment. Instead of directly connecting to the pods to interact with the application services are used.
- **container Runtime :-**

  The container runtime is the software that is responsible for running containers.
  Kubernetes supports several container runtimes: Docker, containerd, cri-o, rktlet and any implementation of the Kubernetes CRI (Container Runtime Interface).

K8 API Primitives / K8 Objects

- K8 api primitives are also called K8 objects.
- Data objects that represent that state of the cluster.
- Example Pod, node, Service, ServiceAccount….
- Everything we do in cluster is represented by some object.
- Every k8 objects includes two sets of fields :-
  spec:- Describes the desired state of the object. Provided by the object creator.
  status:- Actual state of the object, maintained by K8
- Created using declarative method or imperative method.
- Declarative :- Using yaml , json configuration file
- Imperative :- Extensive use of kubernetes command line interface tool kubectl.
- Kubectl :- CLI tool to create cluster , deploy and manage application.
  kubectl software needs to be installed.
- UID :- a system generated string that uniquely identifies the object

Kubectl basic commands

- Print k8 api-resources:

```
cloud_user@ip-10-0-1-101:~$
cloud_user@ip-10-0-1-101:~$ kubectl api-resources
NAME                              SHORTNAMES   APIGROUP                       NAMESPACED   KIND
bindings                                                                      true         Binding
componentstatuses                 cs                                          false        ComponentStatus
configmaps                        cm                                          true         ConfigMap
endpoints                         ep                                          true         Endpoints
```

- Get details of individual objects:

```
cloud_user@ip-10-0-1-101:~$ kubectl get pods
No resources found.
cloud_user@ip-10-0-1-101:~$ kubectl get pods --all-namespaces
NAMESPACE     NAME                                READY   STATUS    RESTARTS   AGE
kube-system   coredns-54ff9cd656-jmbbr            1/1     Running   0          86m
kube-system   coredns-54ff9cd656-wcxnn            1/1     Running   0          86m
kube-system   etcd-ip-10-0-1-101                  1/1     Running   0          85m
```

- Config file for the object can also be obtained using get. Aliasing k=kubectl henceforth.

```
cloud_user@ip-10-0-1-101:~$ k get nodes
NAME              STATUS   ROLES    AGE    VERSION
ip-10-0-1-101     Ready    master   108m   v1.13.3
ip-10-0-1-102     Ready    <none>   108m   v1.13.3
ip-10-0-1-103     Ready    <none>   108m   v1.13.3
cloud_user@ip-10-0-1-101:~$ k get node ip-10-0-1-101 -o yaml
apiVersion: v1
kind: Node
metadata:
  annotations:
```

K8 Pod

- Smallest unit of deployable work in k8.
- Contains one or more containers.
- Containers within pod share IP , port space and can use localhost to communicate with each other.
- Do not have a well-defined life cycle.
- Usually one container per pod is followed but multicontainer pod design pattern exists.
- A PodSpec has a restartPolicy field with possible values Always, OnFailure, and Never. The default value is Always
- Use a Job / cronJob for Pods that are expected to terminate. Restart policy will be OnFailure.
- Use a ReplicationController, ReplicaSet, or Deployment for Pods that are not expected to terminate. Restart policy Always. No need to specify restart policy.
- Use a DaemonSet for Pods that need to run one per machine.

Pod Creation: Declarative

- Creating pod using yaml file

```
apiVersion: v1
kind: Pod
metadata:
   name: pod1
   labels:
     run: pod1
spec:
   containers:
   - image: nginx
     name: pod
     ports:
     - containerPort: 80
cloud_user@ip-10-0-1-101:~$
```

- Creating the pod

```
cloud_user@ip-10-0-1-101:~$ k create -f pod.yaml
pod/pod1 created
cloud_user@ip-10-0-1-101:~$ k get pods
NAME   READY   STATUS    RESTARTS   AGE
pod1   1/1     Running   0          3s
cloud_user@ip-10-0-1-101:~$
```

Pod Creation: Imperative

- Kubectl command to create pod.

```
cloud_user@ip-10-0-1-101:~$ k run pod2 --image nginx --port 80 --restart Never
pod/pod2 created
cloud_user@ip-10-0-1-101:~$ k get pods
NAME   READY   STATUS              RESTARTS   AGE
pod1   1/1     Running             0          70s
pod2   0/1     ContainerCreating   0          6s
cloud_user@ip-10-0-1-101:~$ k get pods
NAME   READY   STATUS    RESTARTS   AGE
pod1   1/1     Running   0          73s
pod2   1/1     Running   0          9s
cloud_user@ip-10-0-1-101:~$ k get pods -o wide
NAME   READY   STATUS    RESTARTS   AGE   IP           NODE           NOMINATED NODE   READINESS GATES
pod1   1/1     Running   0          77s          .5           -102   <none>           <none>
pod2   1/1     Running   0          13s          .2           -103   <none>           <none>
```

- Kubectl command to get details of the pod.

```
cloud_user@ip-10-0-1-101:~$ k get pods
NAME   READY   STATUS    RESTARTS   AGE
app    2/2     Running   0          6s
pod1   1/1     Running   0          17m
pod2   1/1     Running   0          15m
cloud_user@ip-10-0-1-101:~$ k get pods -o wide
NAME   READY   STATUS    RESTARTS   AGE   IP           NODE           NOMINATED NODE   READINESS GATES
app    2/2     Running   0          41s          .4           -103   <none>           <none>
pod1   1/1     Running   0          17m          .5           -102   <none>           <none>
pod2   1/1     Running   0          16m          .2           -103   <none>           <none>
cloud_user@ip-10-0-1-101:~$ k get pods --show-labels
NAME   READY   STATUS    RESTARTS   AGE   LABELS
app    2/2     Running   0          51s   app=my-app
pod1   1/1     Running   0          17m   run=pod1
pod2   1/1     Running   0          16m   run=pod2
```

Get pod examples with labels

- Get pod with specific label :

```
cloud_user@ip-10-0-1-101:~$ k get pods -l run=pod1
NAME   READY   STATUS    RESTARTS   AGE
pod1   1/1     Running   0          19m
cloud_user@ip-10-0-1-101:~$ k get pods -l run=pod2
NAME   READY   STATUS    RESTARTS   AGE
pod2   1/1     Running   0          18m
```

- Get config file of object pod in yaml format :

```
controlplane ~ ➜ k get pod/pod1 -o yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: "2023-06-30T04:38:59Z"
  labels:
    run: pod1
  name: pod1
  namespace: default
  resourceVersion: "4536"
  uid: 53af5fb5-20bc-44a6-82ee-e31181e05ed1
spec:
  containers:
  - command:
```